

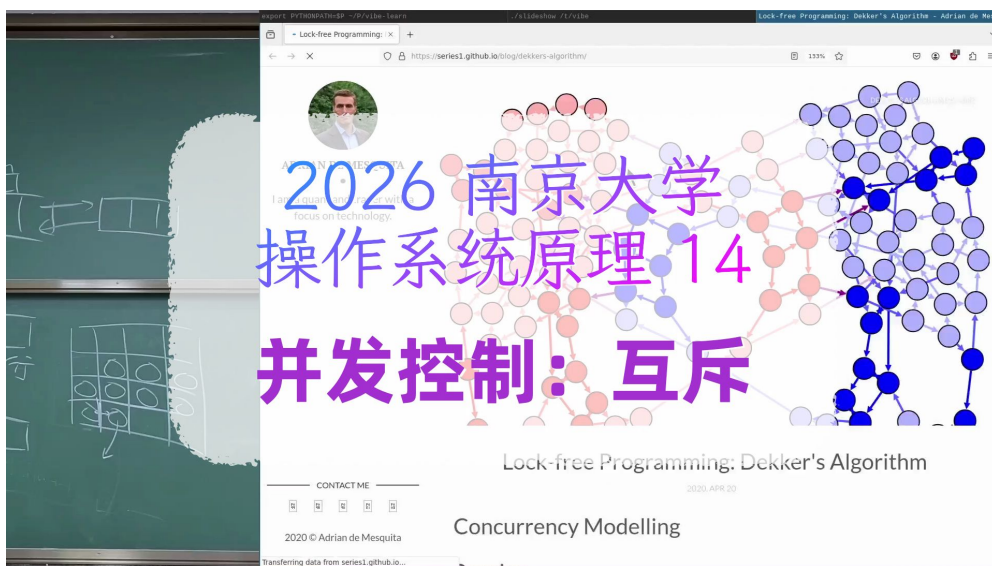
课程笔记

并发控制：互斥

2026 南京大学《操作系统原理》第 14 讲

lecture-to-notes

2026 年 4 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-04-16

视频时长：1:25:38

视频链接：<https://www.bilibili.com/video/BV1hNdhB1Efe/>

目录

1 本讲定位：给并发的魔鬼戴上笼头	2
1.1 为什么操作系统课要讲并发？	2
1.2 目标：一个 <code>make_it_work</code> 的 API	2
1.3 单处理器内核的”作弊”：关中断	3
1.4 本章小结	3
2 互斥 API 的正确使用：Stop the World	4
2.1 互斥锁的两种直观解读	4
2.2 Stop the World：互斥锁的核心直觉	4
2.2.1 一个常见的错误：用了不同的锁	5
2.3 锁的粒度：一把大锁 vs 细粒度锁	5
2.3.1 要么就一把大锁	5
2.3.2 细粒度锁：双向链表的 hand-over-hand locking	5
2.4 互斥与并行：真的冲突吗？	6
2.5 本章小结	6
3 从 Peterson 算法到原子指令	7
3.1 Peterson 算法 (1981)：纯软件互斥	7
3.1.1 Peterson 算法的代码	7
3.2 Peterson 的致命假设	8
3.3 证明 Peterson 的正确性：Model Checking	9
3.4 硬件 rescue：原子指令	10
3.4.1 Spinlock：用原子交换实现互斥	10
3.5 本章小结	11
4 Spinlock 的性能问题与 futex	12
4.1 Spinlock 的两个致命缺陷	12
4.2 从 spinlock 走向 syscall-based lock	12
4.3 futex：fast-path user + slow-path kernel	13
4.4 定量 benchmark：到底哪种锁更快？	13
4.5 Benchmarking crimes：不要做”科研犯罪”	14
4.6 本章小结	14
5 总结与延伸	16
5.1 讲者核心观点	16
5.2 自测三问	16
5.3 与课程其他部分的 synthesis	16
5.4 拓展阅读	16

1 本讲定位：给并发的魔鬼戴上笼头

上一讲（第 13 讲）把并发最可怕的一面摊开来：`sum++` 算不对、`while(!flag)`；变死循环、x86 会观察到 (0,0)。本讲开始给这个魔鬼**戴笼头**——引入互斥（mutual exclusion）这个最重要的并发控制原语，让多线程程序在概念上**退回到顺序执行**，重新变得可理解。

本讲的单句总结

`lock / unlock` 是一段代码能够 “*stop the world*” 的承诺——只要你正确使用，`lock` 和 `unlock` 之间的所有代码就可以当作顺序执行的单线程代码来理解，编译器和 CPU 的那些乱七八糟的乱序重排对你都不再重要。

1.1 为什么操作系统课要讲并发？

UNIX 以来，**进程之间不共享内存**——`fork` 出来的副本独立运行。但有一个地方例外：当进程执行 `syscall`、陷入内核的时候，它们**就都跑在共享的内核代码和数据结构上了**。两个进程同时 `read/write` 同一个文件描述符、同一个 `inode`，它们的内核态部分就变成了需要并发控制的同一份代码。

操作系统

很多早期并发研究的结果（Dekker、Peterson、锁、信号量）都来自操作系统研究者，因为他们**不得不先面对这个问题**。所以今天在操作系统课里讲并发，是“顺理成章”的——这是并发问题最早、最工业化的战场。

1.2 目标：一个 `make_it_work` 的 API

上一讲的教训是：“`sum++` 在多线程里算不对”。我们想要的 API 其实很朴素：

```
1 make_it_work {
2     sum++;           // 或者任意一段长代码
3     sum++;
4     arr[i] = ...;
5 }
```

Listing 1: 理想中的 `make_it_work`

- 如果两个线程同时进入 `make_it_work` 块，一个**必须等**；
- 块内的所有读写**要么全部生效，要么全部不生效**——原子性；
- 块外的多线程并行**照旧高效**。

这不就是数据库的 **transaction**？所以 1990 年代起就有了“Transactional Memory”（事务内存）——Intel 曾在 Haswell 加入 `xbegin / xend` 指令，ARM 也短暂加入过。但两家后来都在消费级产品上**砍掉了这个特性**——它跨越了太多指令，对 CPU 实现（尤其是 cache 一致性、安全漏洞）带来了大麻烦。

1.3 单处理器内核的”作弊”：关中断

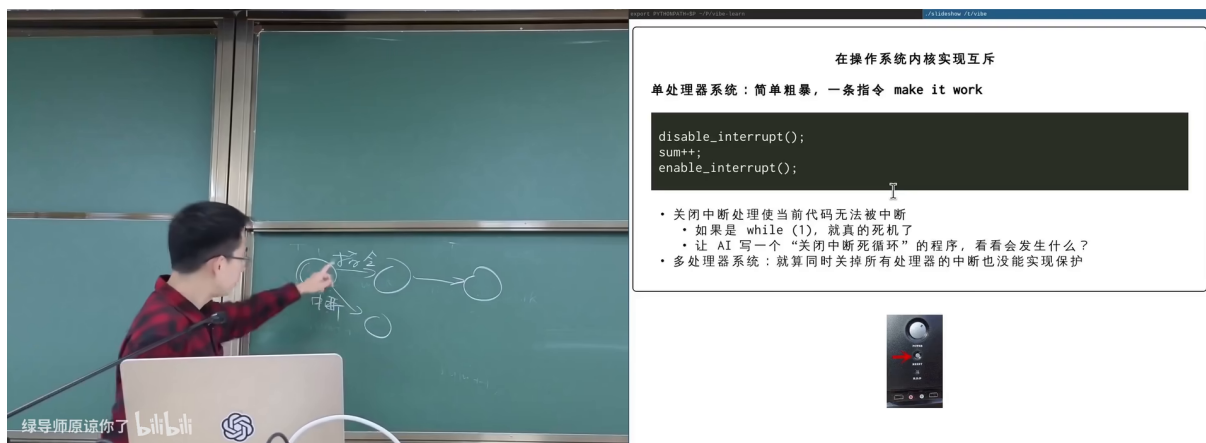


图 1: 单处理器内核实现 `make_it_work` 的最简方法: `disable_interrupt(); sum++; enable_interrupt();`——关了中断就不会被抢占, 也就没有其他线程能打扰你。用户态当然不能这样用——`cli/MSR` 指令在用户态要么 `illegal instruction`, 要么 `segmentation fault`¹

关中断写死循环

单核关中断确实达到”stop the world”, 但你一旦在临界区里写了 `while(1);`, 这台电脑就卡死了 (Windows 蓝屏就是这个年代家常便饭)。多核系统里关中断甚至不够——其他 CPU 核还是能继续运行、继续访问共享内存。所以关中断只是单核内核的权宜之计, 不是通用方案。

1.4 本章小结

互斥的 API 目标很清晰: 在一段代码前后加 `lock()` / `unlock()`, 让这段代码在语义上退回到顺序单线程。实现这个 API 比想象的难——下面几章我们从最简单到工业级逐步构建它。

¹ 视频画面时间区间: 00:07:30–00:07:45。

2 互斥 API 的正确使用：Stop the World

2.1 互斥锁的两种直观解读

1. ”房间 + 门”：房间里只能容纳一个人，lock 是进房间并锁门；unlock 是开门出去。
2. ”桌上的钥匙”：桌面上只有一把钥匙，lock 是把钥匙拿起来；unlock 是把钥匙放回桌面。

两种解读对应的术语就是 **acquire**（获取）与 **release**（释放）——你会在 C11 / C++11 内存模型里反复看到这两个词。

unlock 比 release 多一条保证

unlock 除了”放回钥匙”，还隐含把这个临界区里所有 **store** 全部刷到内存——这是 *release semantics*。相应地 lock 是 *acquire semantics*：不仅拿钥匙，还确保拿锁之后能读到上一个 unlock 之前写入的所有 store。这是并发最容易出错的地方，但好在你用高层 API 时编译器/库帮你填好了。

2.2 Stop the World：互斥锁的核心直觉

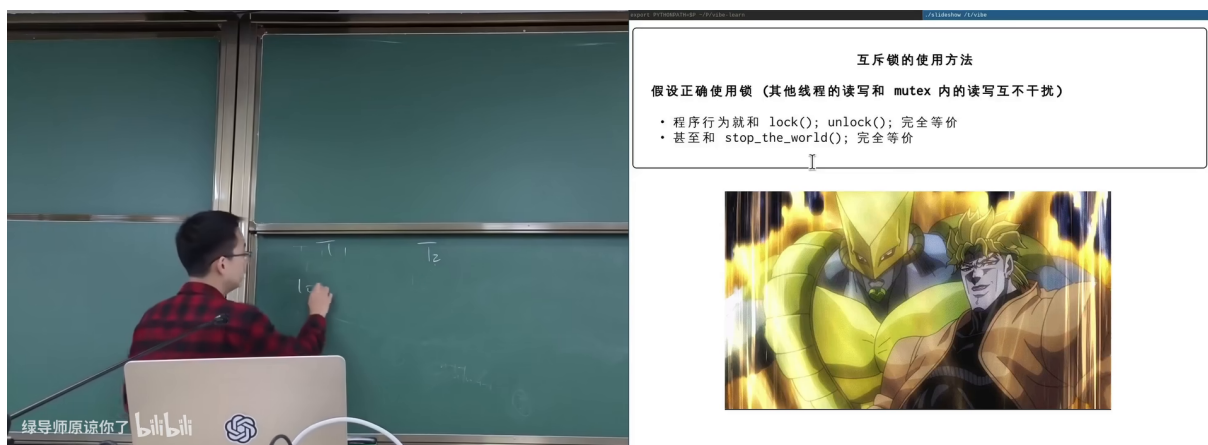


图 2：正确使用 lock/unlock 之后，mutex 的两个不同临界区互不干扰——你可以认为自己 lock 的瞬间 “the world” 就被按下暂停键了。讲者借用了 JoJo 的奇妙冒险中 DIO 的 *Za Warudo* 能力来类比²

这个直觉为什么合法？原因在”正确使用”这个假设——如果你把所有会被并发访问的共享变量都包在同一把锁内外，那从你的临界区视角看：

- 读到什么值 = 上一个 unlock 之前写进去的值，不会被中途改；
- 写进去的值 = 下一个 lock 进来的人一定能读到；
- 其他线程的临界区在时间上可以挪到你前面或后面，结果都不变——所以可以假设”你执行时世界静止”。

²视频画面时间区间：00:19:45–00:20:00。

2.2.1 一个常见的错误：用了不同的锁

```
1 lock_t L, I;           // 两把锁，两个变量名
2
3 void T1() { lock(&L); sum++; unlock(&L); }
4 void T2() { lock(&I); sum++; unlock(&I); } // BUG
```

Listing 2: 看似都加了锁，实际上形同虚设

两个线程分别持有 L 和 I——这两把锁互不知道对方存在，于是 `sum++` 还是 race。”**保护同一个变量就必须用同一把锁**”这条规则看起来简单，但在几十个文件、几十把锁的代码库里非常容易搞错。Linux 内核有专门的运行时工具（lockdep）来检测这类问题。

2.3 锁的粒度：一把大锁 vs 细粒度锁

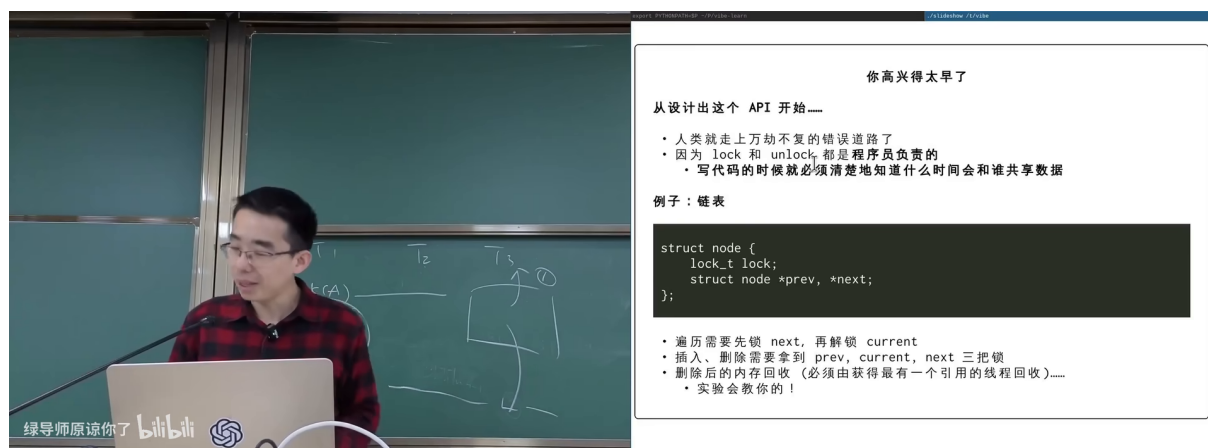
2.3.1 要么就一把大锁

建议（尤其初学者）：只要没有严重性能需求，就用一把大锁保护一切。这是最不容易错的方案。

Big Kernel Lock: Linux 也这样做过

Linux 从 2.0 到 2.6 时代有一个著名的 BKL (**B**ig **K**ernel **L**ock)。早期从单处理器走向多处理器时，Linus 没有一下子把所有子系统的锁都细化——这样做会引入大量 bug 导致内核长期不可用。他用一把大锁把整个内核包起来，然后**按性能瓶颈优先级**慢慢拆分。BKL 要到 2.6.39 (2011) 才被完全移除——花了将近 10 年。

2.3.2 细粒度锁：双向链表的 hand-over-hand locking



你高兴得太早了

从设计出这个 API 开始——

- 人类就走上万劫不复的错误道路了
- 因为 `lock` 和 `unlock` 都是程序员负责的
 - 写代码的时候就必须清楚地知道什么时间会和谁共享数据

例子：链表

```
struct node {
    lock_t lock;
    struct node *prev, *next;
};
```

- 遍历需要先锁 `next`，再解锁 `current`
- 插入、删除需要拿到 `prev`, `current`, `next` 三把锁
- 删除后的内存回收（必须由获得最后一个引用的线程回收）.....
 - 实验会教你的！

图 3: 并发双向链表的正确实现: `struct node { lock_t lock; node *prev, *next; }`。遍历时必须同时持有 `prev`、`current`、`next` 三把锁，以 **hand-over-hand**（像爬梯子一样交替）方式换锁；内存回收时又要确保再没人持有指向该节点的引用——非常容易错³

³视频画面时间区间：00:24:45–00:25:00。

Knuth 的忠告

"Premature optimization is the root of all evil." ——Donald Knuth。在你没证明性能是瓶颈之前，不要为了“一把大锁可能慢”而拆锁。先大锁跑正确、profile 找到真实瓶颈、再针对性拆分——这是 Linux 内核走过 10 年的正确路径。

2.4 互斥与并行：真的冲突吗？

乍看上去矛盾：互斥是“大家排队”，可并行是“同时跑”，不是互相拆台？答案在两条经典定律：

- **Amdahl's Law**: 如果程序有 p 比例可并行, $1-p$ 不可并行, 那么 N 核加速上限是 $\frac{1}{(1-p) + p/N}$ 。即使 $p = 0.5$, 无论多少核最多也只能提速 2 倍。
- **Gustafson's Law**: 如果问题规模可以随处理器数量扩大, 且绝大部分是可并行的, 那么近乎线性加速仍然可达。

实际系统的局部性拯救了一切

物理世界是局部的——每块数据绝大部分时候只被一个线程使用。想象图书馆：每人拿一本书看（并行），只有抢书的瞬间需要锁（互斥）。物理模拟、机器学习训练、搜索、Web 服务器都具备这种结构——所以“互斥控制关键数据结构 + 并行做计算”在工程上完全可行。

2.5 本章小结

lock/unlock 的正确姿势：一把锁、保护所有相关变量、不在临界区做 I/O 或阻塞、unlock 对称；大锁优先，只在必要时细化。理解它不需要记住汇编，只要记住“stop the world”的直觉。

3 从 Peterson 算法到原子指令

那么我们在实现层面，到底怎么才能做出一把锁？本节走一遍从纯软件方案（Peterson）到硬件支持（原子指令）的历史路径。

3.1 Peterson 算法（1981）：纯软件互斥



图 4: 课堂上的 **Peterson 协议** live demo: 两位同学 (Alice 和 Bob) 手持对方的名字标签。要进临界区的步骤: (1) 举手 (store flag); (2) 把手上对方的名字贴到门上 (store turn); (3) 观察对方有没有举手 (load flag); (4) 如果对方举了手, 看门上是不是自己的名字 (load turn); 是则可以进——这就是 Peterson 算法⁴

为什么把对方的名字写到门上？

Peterson 协议里“写对方名字”是个反直觉的设计：如果只有我举手，我可以直接进；如果两人都举手，则由门上的 turn 决定——后写的会覆盖先写的，相当于“我在礼让，让对方先进”。这个协议在 1981 年才被发现是 2 线程互斥最简洁的正确解——之前 16 年里的大部分“正确”算法都是错的。

3.1.1 Peterson 算法的代码

```
1 int flag[2] = {0, 0};
2 int turn;
3
4 void lock(int self) {
5     flag[self] = 1;           // self: 0 or 1
6     turn = 1 - self;         // 举手
7     while (flag[1 - self] && turn == 1 - self) // 把对方名字写到门上
8         ;                    // 对方举了手且轮到他
9 }                             // busy wait
10
11 void unlock(int self) {
12     flag[self] = 0;           // 放手
13 }
```

⁴视频画面时间区间：00:41:45–00:42:00。

3.2 Peterson 的致命假设

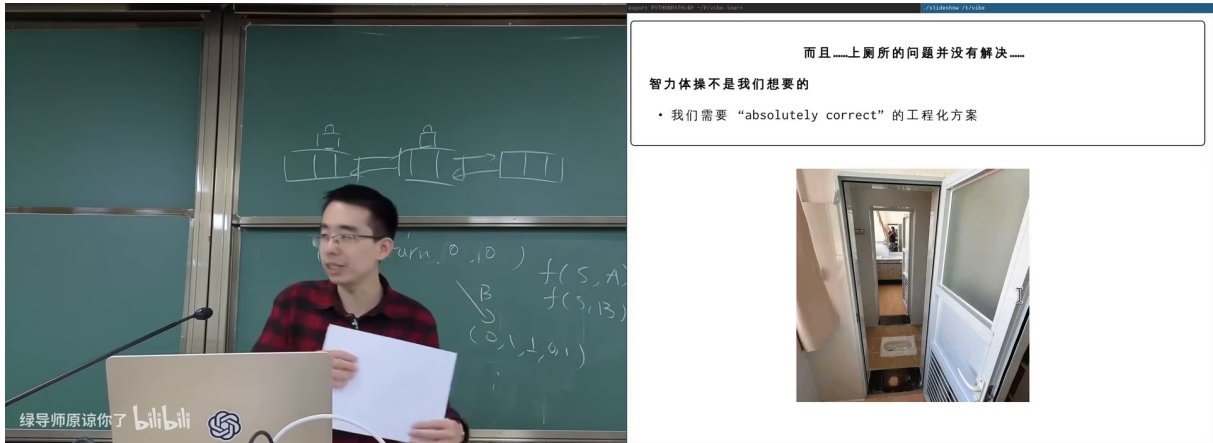


图 5: Peterson 算法的问题：它只对 2 个线程有效；扩展到 N 个线程有方法但代价高，工程上不用；更重要的是——它的正确性依赖两条假设：(1) load/store 指令是原子的、瞬时生效的；(2) 指令按程序书写顺序执行。这两条在现代 CPU 上都不成立⁵

最可怕的是”假装对”的代码

讲者把 Peterson 算法（不加内存屏障）编译后在自己的树莓派上运行——居然跑了很多次都没出错。但这并不意味着代码是对的，只是”机缘巧合”——某次诡异调度就会 crash。这正是并发最可怕的地方：测试通过 $\neq$ 程序正确。你交付给用户之后就惨了。

⁵视频画面时间区间：00:55:45–00:56:00。

3.3 证明 Peterson 的正确性: Model Checking

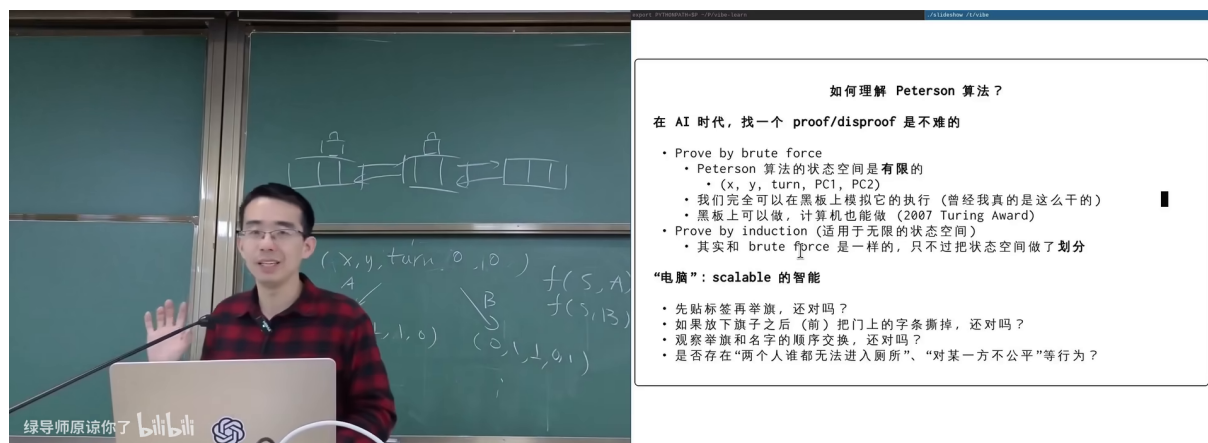


图 6: 如何理解 Peterson 算法? **Prove by brute force**: 把所有 $(x, y, \text{turn}, PC_1, PC_2)$ 状态都枚举一遍, 看是否存在两个线程同时在临界区的状态。这就是 **model checking**——2007 年图灵奖颁给了这一领域的奠基者 Clarke/Emerson/Sifakis。课程里用 Python 写了一个简易 model checker, 可以对各种协议做自动化验证⁶

Systems 人的思维范式

作者反复提到一个 insight: *Computer Science* 的规则是我们自己定的——不像生命科学、物理学那样只能研究既定规律, 我们可以让**硬件或软件配合**来消除问题。

- Peterson 算法试图在 **load/store** 基础上实现锁——硬。
- 不如**改硬件**, 直接给一条能“同时 load + store”的指令——简单得多。

这就是下一节的主题。

⁶视频画面时间区间: 00:49:15–00:49:30。

3.4 硬件 rescue: 原子指令

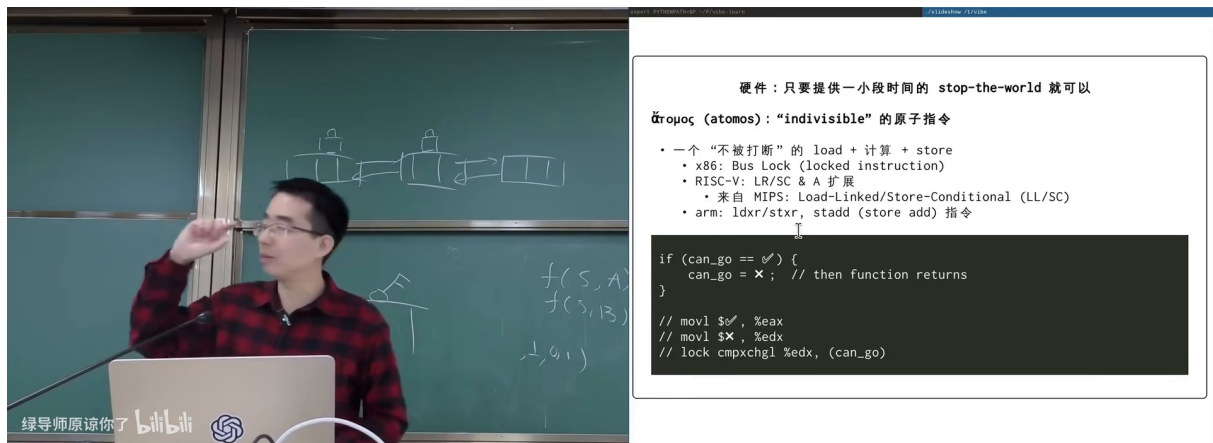


图 7: 硬件提供一小段 stop-the-world 原子性的指令 (**atomic**, 即 “indivisible”): x86: LOCK (bus lock) RISC-V: LR/SC ARM/MIPS: Load-Linked / Store-Conditional (LL/SC) C11: atomic_compare_exchange_strong⁷

X86 的 LOCK 前缀

lock addq \$1, (sum) 相当于: “在执行这条指令期间把总线锁住”——别的核不能访问内存。早期 80486 真的有一个“总线锁定”的物理信号。今天实现方式更精细 (基于 cache line), 但对程序员的可见语义不变。于是 sum++ 的原子版本就简单写成: __atomic_fetch_add(&sum, 1, __ATOMIC_SEQ_CST); 编译到 x86 就是一条 lock addq。

3.4.1 Spinlock: 用原子交换实现互斥

```
1 typedef int spinlock_t;
2 #define LOCKED 1
3 #define UNLOCKED 0
4
5 void spin_lock(spinlock_t *lock) {
6     while (__atomic_exchange_n(lock, LOCKED, __ATOMIC_ACQUIRE) == LOCKED)
7         ; // 拿到的是别人的 LOCKED 标记, 继续 spin
8 }
9
10 void spin_unlock(spinlock_t *lock) {
11     __atomic_store_n(lock, UNLOCKED, __ATOMIC_RELEASE);
12 }
```

Listing 4: Spinlock 实现

核心是 __atomic_exchange_n——把我手上的 LOCKED 和锁里的值原子交换; 如果交换回来的是 UNLOCKED, 说明我抢到了锁。

⁷ 视频画面时间区间: 01:05:00–01:05:15。

3.5 本章小结

纯软件方案（Peterson）理论可行，但假设被现代 CPU 破坏、只支持 2 线程。更好的做法是**让硬件提供一条原子指令**——x86 LOCK、LL/SC、compare-and-swap——spinlock 的实现只用几行代码。但 spinlock 还有自己的问题——下一章解决。

4 Spinlock 的性能问题与 futex

4.1 Spinlock 的两个致命缺陷

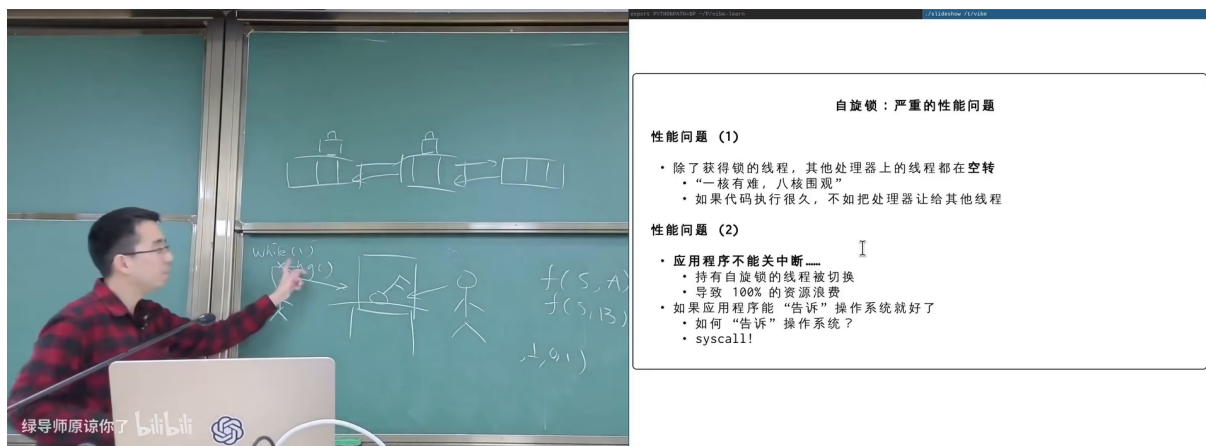


图 8: Spinlock 的性能问题: (1) ”一核有难，八核围观”——一个核拿到锁，其他几个核全部在 `while(atomic_exchange)` 空转，烧掉电却没产出; (2) 更糟——持锁线程可能被操作系统调度出去 (甚至被 `criu` 迁到另一台机器)，剩下所有线程继续 spin 浪费 CPU，直到调度器把它换回来⁸

Spinlock 的适用场景非常狭窄:

- 临界区**极短** (几条指令)，spin 的代价低于上下文切换;
- 持锁线程**不会被抢占** (最典型: 操作系统内核的中断处理路径)。

一旦任一条件不满足，就需要更智能的锁——让**拿不到锁的线程主动去睡觉**。

4.2 从 spinlock 走向 syscall-based lock

要”睡觉然后等锁回来再醒”，线程本身解决不了——必须告诉操作系统。最朴素的做法:

```
1 void lock(int *m) {
2     while (syscall(SYS_mutex_acquire, m))
3         ; // 内核内部会把当前线程标记为 sleep
4 }
5
6 void unlock(int *m) {
7     syscall(SYS_mutex_release, m); // 内核唤醒所有等这把锁的线程
8 }
```

Listing 5: 朴素的 syscall lock (性能差但正确)

每次都进内核

syscall 边界上有**几百纳秒**的固定开销 (trap 到内核、切换寄存器、TLB 刷新、回用户态)。如果 `sum++` 这种 < 10 纳秒的操作每次都陷入内核，开销会增加一百倍以上——见下一节的 benchmark 结果。

⁸视频画面时间区间: 01:08:15–01:08:30。

4.3 futex: fast-path user + slow-path kernel

futex = *fast user-space mutex*, 是 Linux 2002 年引入的一个精妙机制, 将两条路径合并:

- **Fast path** (没人争抢): 纯用户态的 **atomic exchange**, 几纳秒完成 lock/unlock。不进内核。
- **Slow path** (抢不到): 调用 `futex(FUTEX_WAIT, ...)` 让内核把线程挂起; 持锁者 `unlock` 时若发现有等待者, 再调 `futex(FUTEX_WAKE, ...)` 把他们唤醒。

Futex are tricky

futex 的正确实现非常难。最早期 Linus 和 Rusty Russell 公开的示例代码**都是错的**。Rusty 在后来的文章里自嘲: *"I got it wrong the first time as well."* 即使你熟悉内核、熟悉内存模型, futex 也是最容易写错的原语之一。参考阅读: Ulrich Drepper 的《Futexes Are Tricky》。glibc 的 `pthread_mutex_t` 背后就是 futex, 你能安心用它, 是因为有少数专家已经把这些坑填好了。

4.4 定量 benchmark: 到底哪种锁更快?

讲者用一个 **prompt** 让 Claude Code 自动写了 benchmark 程序, 对比:

1. **atomic**: 直接 `__atomic_fetch_add(&sum, 1)`——无锁;
2. **mutex**: `pthread_mutex_lock/unlock` (实际是 futex);
3. **spin**: 自己实现的 spinlock (`atomic_exchange` 版本);
4. **trap**: 每次都强行 `syscall(futex)` 进内核。

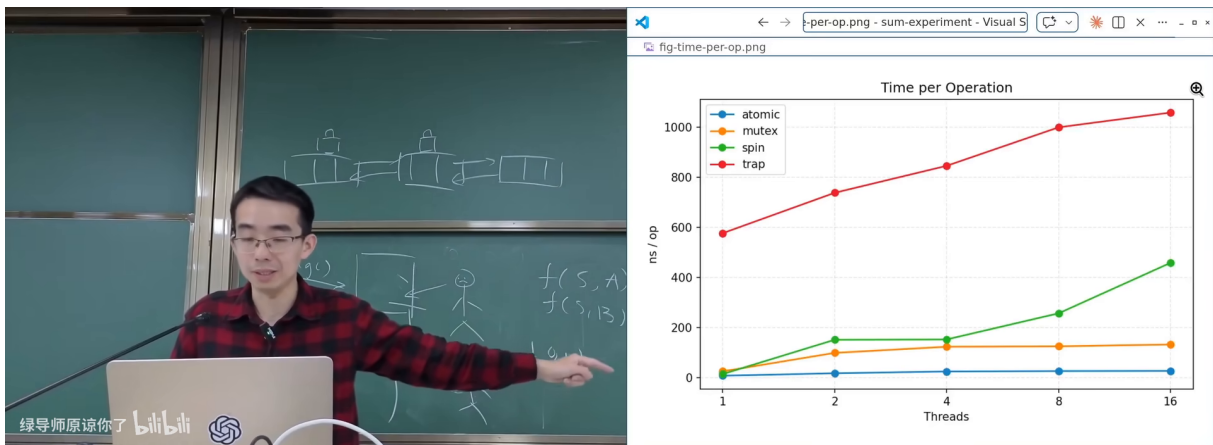


图 9: Time per Operation (纳秒 / 一次 `sum++`) vs 线程数。**Atomic** 在所有线程数下都最快; **mutex** 紧随其后; **spin** 在线程数 $\leq$ CPU 核数时和 **mutex** 差不多, 线程数 $>$ 核数后性能断崖式下跌; **trap** (每次进内核) 的开销始终比其他高一个数量级⁹

⁹视频画面时间区间: 01:19:30–01:19:45。

读图三个 takeaway

1. `atomic` > `mutex` > `spin` > `trap` (性能排序, 多数场景下)。
2. 线程数超过 CPU 核数是 spinlock 崩溃的拐点——印证了前面“持锁线程被抢占就全盘崩溃”的推理。
3. `syscall` 开销是可观的——几百纳秒。这就是为什么 `futex` 要给 `fast-path`。

4.5 Benchmarking crimes: 不要做”科研犯罪”

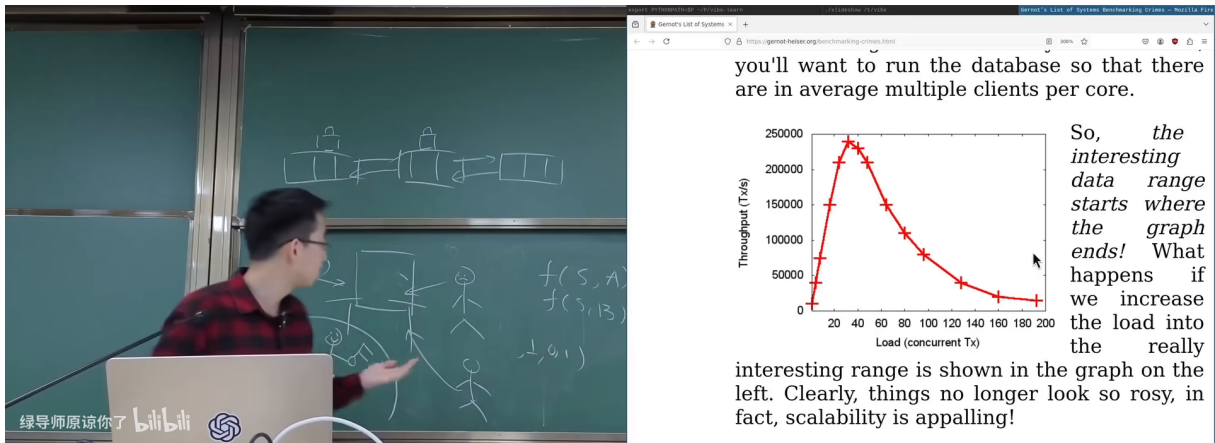


图 10: **Selective dataset hiding deficiencies**——一类 benchmarking crime。图上展示的 throughput 曲线在低 concurrency 时漂亮上升,但在 ~50 clients 处达到峰值后暴跌;如果论文只画左半段就”scalability 惊人”,但去掉右半段就是在”隐藏真实的 scalability 瓶颈”¹⁰

几条经典的 benchmarking crimes

- **Cherry-picking**: 跑 1000 次只报告最好的那次——应该报告均值 + 方差。
- **Selective dataset**: 截断测试范围来隐藏劣化点。
- **不可复现**: 不提供脚本、不提供硬件配置、不提供随机种子。
- **将测试集混进训练集**——讲者调侃某些机器学习圈的”不成文规矩”。
- **将 baseline 不做任何优化**——自己 code 跑满优化, baseline 跑 -00。

Systems 领域目前还没有这种不成文规矩,同学们定量研究时务必先读 **Systems Benchmarking Crimes** 清单。

4.6 本章小结

互斥锁的实现路径:

- 硬件: 原子指令 (x86 `LOCK`、`LL/SC`、`cmpxchg`) ——几纳秒。

¹⁰ 视频画面时间区间: 01:23:30-01:23:45。

- 用户态：spinlock（基于原子交换）——适合短临界区、不被抢占的场景。
- 用户 + 内核：futex——无争抢时走用户态 fast-path、争抢时走内核 slow-path。
- pthread_mutex_t 就是 futex 的封装——对你来说，**直接用它就是标准做法**。

性能工程的正确姿势永远是：**先写对，再 profile，再针对真实瓶颈优化**。

5 总结与延伸

5.1 讲者核心观点

1. **lock/unlock 是并发世界的”stop the world”**——正确使用后，临界区可以当作顺序程序理解，13 讲里那些魔鬼都消失了。
2. **”保护同一个变量用同一把锁”**看起来简单，是绝大多数并发 bug 的根源。
3. **大锁优先原则**——先把程序写对；只在真实性能瓶颈上才拆成细粒度锁。Linux 花了 10 年才把 BKL 移除，你不需要比它更激进。
4. **Systems 人的解题思路**：不要局限于既有规则——*Peterson* 算法是硬来软的好例子、失败的典型；原子指令是改硬件的好例子、成功的典型。
5. **Spinlock 的缺陷暴露了一个边界**：线程数 > CPU 数时会崩溃。所以有了 futex——fast path 用户态、slow path 内核。
6. **测试通过 ≠ 正确**——并发 bug 可能”机缘巧合”潜伏很久，交付后才爆发。用 model checker 做形式化验证是更靠谱的路子。
7. **Benchmarking crimes** 是科研的高压线——Systems 领域做任何定量结论前先读一遍清单。

5.2 自测三问

看完本讲，能答上这三题吗？

1. 在单核内核里用 cli/sti 实现 make_it_work 为什么够用？多核为什么不够？
2. 写一个会观察到 Peterson 算法在 x86 上失效的程序：用 __atomic 替换 volatile，观察 assert(0) 是否触发。
3. 为什么 spinlock 在线程数 > CPU 数时会崩溃？如果你一定要在这种情况下用 spin（比如硬实时场景），怎样缓解？

5.3 与课程其他部分的 synthesis

- 与第 13 讲（入门到放弃）：13 讲展示并发问题，14 讲给第一个解决方案——互斥。但互斥不是万能的；后续会引入条件变量、信号量、无锁数据结构、RCU...
- 与第 12 讲（Linux from scratch）：12 讲里 pivot_root 完成了”系统从无到有”的叙事；14 讲里 futex 是 pthread_mutex_t 的内核侧实现——你熟悉的 API 底层都是 syscall。
- 与后续并发控制章节：互斥只是开始。接下来会看到：条件变量（等待 + 通知）、信号量（计数的互斥）、读写锁（区分 reader/writer）、无锁数据结构（CAS-based）、RCU（无读者开销的读多写少场景）。

5.4 拓展阅读

- Peterson, *Myths about the Mutual Exclusion Problem*, Information Processing Letters, 1981.

- Ulrich Drepper, *Futexes Are Tricky*, Red Hat Whitepaper, 2011. <https://www.akkadia.org/drepper/futex.pdf>
- Linux kernel 文档: `Documentation/locking/mutex-design.rst`, `Documentation/locking/spinlocks.rst`, `man 2 futex`.
- Herlihy & Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2008 ——并发数据结构的经典教材。
- Heiner *et al.*, *Systems Benchmarking Crimes*, ZU collection/workshop 2021+ ——做 systems 实验前必读。 <https://www.cse.unsw.edu.au/~gernot/benchmarking-crimes.html>
- Clarke & Emerson, *Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic*, 1981 ——model checking 奠基论文；2007 图灵奖。
- `man 3 pthread_mutex_lock`、`man 7 pthreads`、`__atomic_*` builtins (GCC Manual)。